

Spatial Informatics (GS2.401)

Quiz 2 Comprehensive Study Guide

Lectures: 4 Sept – 19 Sept 2026

Data Cleaning & Topology · Vector Data Analysis · Raster Data Analysis

Compiled from slide decks, transcripts, handwritten notes & PYQs

Contents

How to Use This Guide	2
1 4 Sept & 8 Sept: Data Cleaning, Topology and Generalization	3
1.1 Overview	3
1.2 Primary Data Capture Errors	3
1.3 Secondary Data Capture Errors (Digitizing)	3
1.4 Vectorization Without Post-Processing	4
1.5 Georeferencing with Imprecise Control Points	4
1.6 Transformation, Rubber-Sheeting, Edge-Matching, Warping	5
1.7 Data Entry Errors & Image Interpretation Error	5
1.8 Object Relationships	6
1.9 Topological Relationships Between Spatial Objects	6
1.10 Relationships as Attributes – Worked Examples	6
1.11 Topology: Definition, Properties, Advantages	7
1.12 Arc-Node Topology	8
1.13 Map Scale	9
1.14 Map Generalization	9
1.15 Spatial Inference	10
1.16 Querying Spatial Data	10
1.17 Practice & Review	10
2 15 Sept: Vector Data Analysis	12
2.1 Overview	12
2.2 Boolean Logic in GIS	12
2.3 Classification of Vector Overlay Operations	12
2.4 Point-in-Polygon: The Ray-Casting / Odd-Even Rule	12
2.5 Worked Overlay Examples (from slides)	13
2.6 Clip vs. Intersect vs. Union vs. Difference – THE Comparison Table	13
2.7 Data Manipulation: Reclassify → Dissolve & Merge	14
2.8 Buffering	14
2.9 Vector Geoprocessing Tool Reference	15

2.10	Case Study: Flood-Exposed Villages, Schools & Roads	15
2.11	Practice & Review	15
3	18 Sept: Raster Data Analysis – Part 1	17
3.1	Overview	17
3.2	Raster Functions/Operations – The Big Picture	17
3.3	Raster-Based Overlay	17
3.4	Overlay: The 4 Basic Rules	17
3.5	Cell/Pixel/Grid-Based Operation Classification	18
3.6	Local Functions	18
3.7	Reclassification	19
3.8	Practice & Review	19
4	19 Sept: Raster Data Analysis – Part 2	21
4.1	Overview	21
4.2	Focal Functions (Neighbourhood)	21
4.3	Neighbourhood Analysis / Spatial Filtering	21
4.4	Zonal Functions	22
4.5	Global (Incremental) Functions	23
4.5.1	Proximity	23
4.6	Practice & Review	24
5	Previous-Year Question Bank (Filtered & Adapted)	25
5.1	Multi-Select Style (2025 paper Part I) – watch for booby-trap options	25
5.2	Scenario / Overlay-Type Identification (2025 paper Part II)	26
5.3	Applied/Design Questions (2024 paper, topics in scope)	28
6	Master Cheat Sheet	30
7	Rapid Self-Test	31

How to Use This Guide

This guide covers the five class sessions on your quiz syllabus: **4 Sept** and **8 Sept** (both from the slide deck “*Class 4: Data Cleaning, Topology and Generalization Techniques*”), **15 Sept** (“*Class 5: Vector Data Analysis*”), and **18 Sept + 19 Sept** (both from “*Class 6: Raster Data Analysis*”, Parts 1 and 2). Content is built directly from the slide decks (the primary source of exam questions per the instructor’s own pattern), cross-checked against your handwritten notes (which show the same topic tree), and supplemented by the lecture-transcript summaries where they add clarifying detail. Note: the 8-Sept transcript/summary files were not available (no audio was transcribed for that session) – that day’s content is reconstructed purely from its slide decks (Part 2 and Part 3 of Class 4) and classwork, which is why Sections 1 treats 4 Sept and 8 Sept as one continuous topic (they are literally one slide deck split across two class days).

Section 5 mines two previous mid-semester papers from the *same instructor* (Dr. P. Rama Chandra Prasad) for recurring question patterns – one dated 25 Sept 2025 (covering almost this exact topic range) and one dated 21 Sept 2024 (partially overlapping). Questions on topics *not* covered in your 4–19 Sept slides (map projections, geoid/DGPS, BIP/BSQ/BIL raw raster formats) have been **deliberately excluded** since they will not appear on this quiz even though they are genuine PYQs – only questions whose underlying concept is in your current slide range are kept, and they are reworded/adapted rather than reproduced verbatim.

Study priority (based on slide emphasis + PYQ repetition)

1. **Vector overlay operations** and the Clip/Intersect/Union/Erase/Difference distinction (attribute handling is the classic trick).
2. **Topology**: definition, arc-node topology, the 5 types of topological relationships, and which properties survive a “rubber-sheet” stretch.
3. **Raster operation classification**: Local vs Focal vs Zonal vs Global – expect a “classify this real-world scenario” question.
4. **Point-in-polygon** (ray-casting / odd-even rule) and the **4 overlay rules** (Enumeration, Dominance, Contributory, Interaction).
5. **Data-cleaning error types**: node errors (undershoot/overshoot), polygon errors (sliver, dangling node, pseudonode), and how snapping/splitting fixes them.
6. **Reclassification** (arbitrary, Boolean, statistical/slice) and **Map generalization** operators (simplify, smooth, collapse, aggregate, amalgamate, merge, refine, exaggerate, enhance, displace).
7. Buffer/Proximity, focal neighbourhoods (kernel shapes, mean/median/edge filters), and zonal statistics.

1 4 Sept & 8 Sept: Data Cleaning, Topology and Generalization

1.1 Overview

This is a single slide deck (*Class 4*, split across two class days) with three big themes: (1) where errors enter a GIS dataset and how to fix them, (2) object relationships and topology, and (3) scale, generalization, and querying spatial data.

1.2 Primary Data Capture Errors

Errors that enter *during* data collection (before anything is digitized).

1. **Positional accuracy of GPS receivers.** Sources of GNSS positioning error:
 - **Satellite clock bias** and **satellite orbit error** (ephemeris error)
 - **Ionospheric and tropospheric delay** (signal slows passing through atmosphere)
 - **Multipath** (signal bounces off buildings/trees before reaching receiver)
 - **Receiver clock bias**
 - Depends on the **set of “caught” (visible) satellites**
2. **Device/instrument operator error or malfunction of electronic scanners**, producing three characteristic image defects:
 - **Systematic noise** – regular striping across the image (banding)
 - **Random noise** – salt-and-pepper **speckles** (isolated bad pixels)
 - **Line dropouts** – entire scan lines missing (e.g. Landsat 7 SLC-off gaps); fixed by *gap-filling*, e.g. averaging the preceding and succeeding line’s pixel values
3. **Geometric distortion in remote sensing data** – caused by platform **altitude** and **attitude** changes:
 - *Aerial photography*: change in **altitude** rescales the whole image (nominal → larger/smaller scale); change in **attitude** – **roll** (about the flight-direction axis) causes compression/expansion left-right; **pitch** (nose up/down) causes fore/aft compression-expansion; **yaw** rotates the whole frame (aircraft crabbed into wind).
 - *Satellite imagery*: **cross-track scanner distortion** (pixel size grows off-nadir because the same angular swath covers more ground at the edges); **mirror-velocity variation** (actual vs. nominal mirror angle over time) causing **scan skew**; and, evaluated either from tracking data (Earth rotation, spacecraft velocity) or from ground control (altitude/pitch/roll/yaw variation), each produces a characteristic parallelogram/trapezoid distortion of the nominal square image.

1.3 Secondary Data Capture Errors (Digitizing)

Errors introduced while converting a paper/raster source into vectors by hand.

Node errors – snapping

When digitizing without a proper snapping tolerance you get:

- **Undershoot** – a line stops just short of the node/line it should connect to
- **Overshoot** – a line runs past the node it should stop at

Fixing node errors matters because it **validates connectivity** and **validates contiguity**. The fix is interactive editing inside a proper **snap tolerance** (a small capture radius around each node).

Polygon errors (see the classic “polygon errors” diagram):

- **Sliver polygons** – thin spurious polygons created when two layers that *should* share an identical boundary are digitized slightly differently and then overlaid.
- **Dangling node** – a node at the free (unconnected) end of a line that should have closed a polygon.
- **Pseudonode** – a node where only two arcs meet and both belong to the same feature (i.e. a redundant vertex splitting one logical line into two arcs with no real topological meaning). Removed with the **split/unsplit** tools.
- **Missing label point** – a polygon with no attribute label point inside it.
- **Line crossing with no node** – two lines cross on the map but no node was placed at the intersection, so the software does not know they are topologically connected.

Editing tools to fix these: add missing arc, correct overshoot/undershoot, fix an open polygon, split & unsplit, move & add vertex, add/delete/change labels.

1.4 Vectorization Without Post-Processing

Directly vectorizing a classified raster (e.g. Forest/Agriculture/Built-up/Water/Barren) without cleanup produces: **over-segmentation** (too many tiny polygons), **spikes and sliver polygons**, **gaps between polygons**, **dangling polygons**, and **irregular boundaries**.

Key point: Post-processing (cleaning, smoothing, editing) is *essential* for quality vector data – raw vectorization is never exam-ready output.

1.5 Georeferencing with Imprecise Control Points

If the Ground Control Points (GCPs) picked on the source map do not precisely match their true location on the reference map, the whole georeferenced output is displaced/distorted – shown as an **overall shift** plus per-point **positional error**. Quality is checked via the **RMS (root-mean-square) error** of the GCP residuals; a well-distributed set of GCPs (not clustered) with low RMS is required for a trustworthy transform.

RMS error of GCP residuals

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N [(\Delta X_i)^2 + (\Delta Y_i)^2]}$$

where $\Delta X_i, \Delta Y_i$ are the differences between a GCP's true coordinate and its transformed coordinate. Lower RMS = better fit; a common target is $\text{RMS} < 1$ pixel.

1.6 Transformation, Rubber-Sheeting, Edge-Matching, Warping

Transformation converts data from one coordinate system to another, or repositions data within a coordinate system: **translation** (shift), **rotation**, **differential scaling**, and **skew**, defined via a **source control point** → **destination control point displacement link**.

- **Rubber-sheeting** – corrects geometric distortions (from imperfect map-compilation registration, lack of geodetic control, etc.) by adjusting a source layer (dashed) onto a target layer (solid) using displacement links and snapping properties – like stretching a rubber sheet pinned at the control points.
- **Edge-matching** – reconciling mismatched boundaries where two adjacent map sheets (source layer / target layer) meet.
- **Warping** – the general operation of applying many displacement links across a whole layer (e.g. warping India's district boundaries from one zoning system to another using dozens of matched vertex pairs).

Practical link: the 8-Sept classwork

The 8-Sept QGIS exercise (two 1950s USGS topo sheets, *Mariposa* and *Yosemite*) is a real-world instance of this section: setting a Project CRS, then choosing a **NAD27→NAD83→NAD83(HARN)** transformation pipeline for each raster before digitizing a common polygon – i.e., *georeferencing* / *coordinate transformation* in practice, not merely as slides.

1.7 Data Entry Errors & Image Interpretation Error

- **Missing attribute values** – caught by comparing the unique list of values in a field against a master list.
- **Incorrect attribute values** – typing errors, caught with a spelling check.
- **Image interpretation / resampling error** – when resampling a raster to a new grid, three interpolation choices trade off speed vs. smoothness:
 - **Nearest neighbour** – copies the closest input cell's value (fastest, blocky, preserves categorical data)
 - **Bilinear interpolation** – weighted average of the 4 nearest cells (smoother, blends values – unsuitable for categorical data)

- **Cubic convolution** – weighted average of the 16 nearest cells (smoothest, most computation)
- **Data transfer:** information loss can be caused by **file conversion** and by **resampling of data**.

1.8 Object Relationships

Two broad kinds of relationship

- **Spatial relationships** – e.g. “is contained in”, “intersects”. Can exist between entities of the *same* type or of *different* types.
- **Relationships as attributes** – a relationship stored as data in a table rather than computed geometrically.

Types of relationship (feature relationships):

1. **Relationships that construct complex objects from primitives** – e.g. a line is defined by its ordered set of points; a polygon is defined by its ordered set of bounding lines.
2. **Relationships computable from coordinates** – e.g. “is contained in” (point-in-area test), “overlaps” (do two areas overlap?).
3. **Relationships *not* computable from coordinates** – e.g. we can compute whether two highway *lines* cross, but not whether the highways they represent actually intersect at grade (it might be an overpass) – that needs extra attribute data.

1.9 Topological Relationships Between Spatial Objects

Pair	Relationships
Point–Point	<i>is within</i> (within a distance); <i>is nearest to</i>
Point–Line	<i>on line</i> ; <i>is nearest to</i> (a line)
Point–Area	<i>is contained in</i> (in area); <i>on border of area</i>
Line–Line	<i>intersects</i> ; <i>crosses</i> (cross without a true topological intersect); <i>flows into</i> (e.g. a stream flows into a river)
Line–Area	<i>intersects</i> (an area); <i>borders</i> (line is part of the area’s border)
Area–Area	<i>overlaps</i> ; <i>is within</i> (an island within an area); <i>is adjacent to</i> (share a common boundary)

1.10 Relationships as Attributes – Worked Examples

“Flows-in” example (stream network). Two equally valid storage options:

- **Option A:** each *link* stores the ID of the *downstream link* it flows into (a Link ID → Downstream table). Flow is traced link-to-link by following the pointer.
- **Option B:** each *link* stores the ID of the downstream *point*, and each *point* separately stores the ID of the downstream *link* – a two-table (Link→Point, Point→Link) pointer chain.

“Is contained in” example. To find, for each well (point), which county (polygon) it lies in: (1) compute the point-in-polygon relationship for every well, (2) store the result as a new attribute **County** on the well table, (3) aggregate (e.g. COUNT wells grouped by **County**) to build a summary table.

Object pairs / distance matrix. Some attributes belong to a *pair* of objects, not a single object – distance between A and B, commuter flow between two places, trade between two counties. These are stored as a **matrix** (rows/cols = objects, cell = the pairwise attribute) or as a directed **arrow attached to an origin→destination link**.

1.11 Topology: Definition, Properties, Advantages

Definitions to quote

- ESRI: “A GIS topology is a set of **rules** and **behaviors** that model how points, lines, and polygons share **geometry**.”
- Longley: “Topology is the science and mathematics of **relationships** used to validate the geometry of vector entities, and for operations such as network tracing and tests of polygon adjacency.”
- In one line: **topology is a mathematics approach that defines unchangeable spatial relationships.**

What survives a stretch/distortion of the map, and what does not:

Changes under stretching	Stays the same (topological properties)
Distance	Adjacency
Angles	Most other relationships (“is contained in”, “crosses”)
Relative proximities	The <i>type</i> of object – areas stay areas, lines stay lines, points stay points

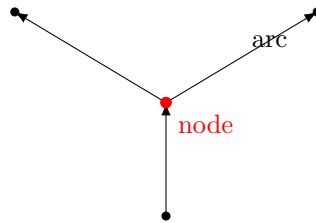
Advantages of topology: (1) efficient data storage and quick processing of large datasets; (2) enables spatial-model operations (network tracing, adjacency tests) that would otherwise need brute-force coordinate geometry every time.

The 5 types of topological relationships (memorize this list – very testable):

1. **Connectivity** – which features connect to / intersect each other.
2. **Adjacency** – which features are contiguous to each other.
3. **Containment (nestedness/insideness)** – which features (node, arc, smaller polygon) are contained within a polygon.
4. **Proximity** – which features are near others.
5. **Relative direction** – the relative position between features.

1.12 Arc-Node Topology

- An **arc** is defined by two endpoints: a **from-node** (where it begins) and a **to-node** (where it ends). Arc-node topology records which arcs are connected at which nodes.
- Supported by an **arc-node list**; connected arcs are found by searching the list for **common node numbers**. A set of connected arcs defines a **network**.
- The from-node/to-node pair also encodes the arc's **direction** – this is what makes representing **flow data** possible (e.g. which way a stream flows).
- **Adjacency / left-right topology**: for each directed arc, the polygon on its left and the polygon on its right can be determined from the arc's direction – two polygons sharing a boundary are *adjacent*.



Common node numbers link arcs into a network

Six named “topology types” shown as a summary graphic on the slides (know these labels, they are quizzable):

- **Arc-node topology** – line features share endpoints.
- **Polygon topology** – area features share boundaries.
- **Route topology** – line features share segments with other line features.
- **Region topology** – area features can overlap with other area features.
- **Node topology** – line features share endpoint vertices with point features.
- **Point events** – point features share vertices with line features.

Why topology matters for GIS analysis – two classic slide examples

1. Shortest path pitfall. If topology at a junction is *poorly defined* (e.g. two crossing yellow line segments are treated as “one” line rather than properly noded), a shortest-path calculation between two points can give the *wrong* answer (the slide example: correct answer 5.5 map units through a properly-noded path vs. an incorrect 6 if the junction topology collapses distinct segments). **2. Adjacency pitfall.** Two overlapped polygons must be **cut into three** distinct polygons (the two non-overlapping parts + the overlap) to unambiguously define which polygons are adjacent to which – otherwise “find the adjacent polygon” queries fail.

1.13 Map Scale

Scale = the relationship between distance on the map and distance on the ground. Expressed three ways: **word statement** (“one inch to a mile”), **Representative Fraction / RF** (e.g. 1:24,000), and **Graphic (bar) scale**.

Category	Typical RF	Detail
Large scale	1:12,000 – 1:25,000	More detail, e.g. a city
Intermediate scale	1:50,000 – 1:100,000	Medium detail
Small scale	1:250,000 – 1:750,000+	Coarser, e.g. a country

Counter-intuitive but exam-favourite fact: “Large scale” = *small* area shown in *great* detail (small denominator, e.g. 1:5,000 for a stadium); “small scale” = *large* area shown coarsely (large denominator, e.g. 1:3,000,000 for all of India). **Scale matters for:** analysis validity, data integration (mixing scales causes error), and visualization.

Impact of scale on representation: the same city can be a *point* at 1:250,000 or an *area (polygon)* at 1:5,000. The same river simplifies from a wiggly line at 1:25,000 to nearly straight at 1:100,000+. Scale also visibly changes DEM-driven **hydrologic simulation** results (a 10 m DEM vs. a 100 m DEM of the same river gives different discharge numbers at the same gauge points).

1.14 Map Generalization

Definition: the process of designing a map to reduce real-world complexity by strategically reducing ancillary/unnecessary detail, while still emphasizing the important elements faithfully and recognizably. Needed because un-generalized data, viewed at a smaller scale, becomes cluttered (“crowding of features”) and the intended spatial pattern/message gets obscured.

The 10 generalization operators (spatial transformations) – know definitions *and* be able to identify one from a picture

- **Simplification** – weed out points from a polygon outline to make a simpler shape.
- **Smoothing** – replace sharp/complex forms with smoother ones.
- **Collapse** – replace an area object with a combination of point and line objects (e.g. a lake becomes a point+outline at small scale).
- **Aggregation** – replace many distinct symbolized objects with fewer new symbols (e.g. scattered “Pueblo Ruins”/“Miguel Ruins” points become one “Ruins” symbol).
- **Amalgamation** – replace *several area objects* with a *single* area object.
- **Merging** – replace several *line* objects with a smaller number of line objects.
- **Refinement** – replace a complex pattern of objects with a selection that preserves the pattern’s general form (e.g. simplifying a dense branching river network but keeping its overall shape).
- **Exaggeration** – relatively enlarge an object to preserve a characteristic (e.g. a bay/inlet) that would otherwise be lost at scale.

- **Enhancement** – alter the physical size/shape of symbols (not the underlying geometry).
- **Displacement** – move objects from their true position to preserve visibility/distinctiveness (e.g. shifting a road symbol off an overlapping railway line).

Generalization of attributes – Classification generalization: reclassifies the attributes of objects into a smaller number of classes (distinct from geometric generalization above).

1.15 Spatial Inference

Sampling shortens data-collection time but leaves gaps that must be filled:

- **Interpolation** – predict missing values *between* two known bounding values (both sides known).
- **Extrapolation** – predict missing values *outside* the bounding area (only one side is known) – inherently less reliable.

1.16 Querying Spatial Data

Query type	Example
Cartographic / Map queries	Find an ATM, identify MG Road, locate a district/zone
Attribute-level queries	Which road is longer – MG Road or RP Road?
Spatial queries	Which is the nearest hospital? Does CR Rao Road intersect a railway line?
Spatio-Attribute queries	Find the nearest ATM with > 4 machines
Topological queries	Which survey numbers get affected by road widening? Is IIIT-H in Hyderabad district?

Worked example: “Select all airports in India that belong to seismic zone V” – overlay the airport point layer on the seismic-zone polygon layer and select points whose containing polygon = Zone V (a point-in-polygon spatial query).

1.17 Practice & Review

Q1. [MC] A vertex that appears twice at the same location, splitting one logical line into two arcs that have no real topological meaning, is called a:

- (A) Dangling node
- (B) Pseudonode
- (C) Sliver polygon
- (D) Overshoot

Answer: (B) Pseudonode – removed via the split/unsplit tool.

Q2. [MC] Which of these survives when a map is stretched/rubber-sheeted?

- (A) Distance between two points
- (B) Angle between two lines
- (C) Adjacency between two polygons
- (D) Relative proximity ranking

Answer: (C) Adjacency (and most other topological relationships like “contains”/“crosses”) is unchanged; distance, angle, and proximity all change.

Q3. [Short Answer] Name the five types of topological relationships. *Answer:* Connectivity, Adjacency, Containment, Proximity, Relative direction.

Q4. [Short Answer] Why must a from-node/to-node arc representation exist for flow modelling? *Answer:* Because direction is encoded in which endpoint is the from-node vs. the to-node; without direction you cannot say water/traffic flows from A to B rather than B to A, so left-right (adjacency) and downstream tracing would be undefined.

Q5. [Short Answer] Distinguish *amalgamation* from *aggregation* as generalization operators. *Answer:* Amalgamation merges several *area* objects into one area object (same object type, fewer instances). Aggregation replaces many distinct *symbolized* objects (which may be points) with a smaller number of new symbols, often changing the symbol’s meaning/type (e.g. several ruin points → one “ruins” label), not necessarily merging geometry.

Q6. [Essay] A district administration digitizes agricultural farm boundaries in a village. State two topological constraints that *should* and one that should *not* apply, and justify. *Answer:* Should apply: (1) two neighbouring farms cannot overlap (else area double-counts); (2) each boundary line segment can belong to at most two neighbouring farms (arc-node/left-right topology – a shared edge has exactly one polygon on each side). Should *not* apply: farms cannot overlap is required, but the reverse claim “farms *can* overlap” would be wrong to enforce as an allowed rule – however corners *can* legitimately be shared by more than two farms meeting at a point (that is a valid node, not a topology violation), so a rule forbidding multi-farm corners would be an incorrect constraint to apply.

2 15 Sept: Vector Data Analysis

2.1 Overview

The world is complex; vector data represents a simplified version of it using points, lines, and polygons. Two procedures get data into a GIS: **selection** (choosing which real-world features to include) and **representation** (choosing point/line/polygon geometry, informed by scale – link back to Section 1). The heart of this lecture is **overlay operations**, grounded in **Boolean set logic** and **computational geometry**.

2.2 Boolean Logic in GIS

- **AND** ($\cap$) – intersection: entities in *both* A and B.
- **OR** ($\cup$) – union: entities in *either* A or B.
- **NOT** ($\neg$) – difference: entities in A but *not* in B.
- **XOR** – entities in exactly one of A or B (symmetric difference).
- Compound expressions are testable: (A AND B) OR C, A AND (B OR C) – these give *different* resulting regions, so bracket order matters.

2.3 Classification of Vector Overlay Operations

Two ways to classify: (1) by the **elements** contained in the layers being overlaid, or (2) by **operation type**.

	Point	Line	Polygon
Points	Points coincide	Point in line	Point in polygon
Lines	Point in line	Line intersection	Line in polygon
Polygons	Point in polygon	Line in polygon	Polygon overlay

2.4 Point-in-Polygon: The Ray-Casting / Odd-Even Rule

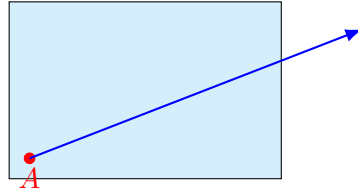
The rule

Cast a ray from the query point A in any fixed direction. **Count how many polygon edges the ray crosses.** An **odd** number of intersections means A is **inside** the polygon; an **even** number means A is **outside**.

For finding where two line segments intersect (needed to build the crossing count), given $y = a_1 + b_1x$ and $y = a_2 + b_2x$:

$$x_i = -\frac{a_1 - a_2}{b_1 - b_2}, \quad y_i = a_1 + b_1x_i$$

then check that x_i falls within the valid x -range of *both* line segments (otherwise the lines' infinite extensions cross, but the actual segments do not).



1 crossing (odd) $\Rightarrow A$ is inside

2.5 Worked Overlay Examples (from slides)

- **Point in polygon:** 5 restaurants (points) overlaid on 3 city wards (polygons) $\rightarrow$ a joined table stating which ward each restaurant falls in.
- **Line in polygon:** a road network overlaid on soil-type polygons $\rightarrow$ each road segment is split at soil boundaries and tagged with (Highway Number, Soil Type).
- **Polygon overlay:** a watershed layer overlaid on a district layer $\rightarrow$ output polygons each tagged with (Watershed ID, District ID) – this is the general case underlying most multi-criteria GIS analysis.

Other overlay use-patterns from the slides:

- **Query** – “what is the area of both A and Y?” (an AND/intersection query across two coverages).
- **Reclassify** – collapse categories (e.g. A and C reclassified into a single new class P) *before* overlay, simplifying downstream analysis.
- **Coverage rebuilding** – clip a coverage onto one zone and erase the rest.
- **Overlay ranking** – overlay two ranked layers (e.g. land-use ranks $\times$ vegetation ranks) and look up the combined rank in a ranking table to get a composite suitability score.
- **Connectivity** – rezonify a buffer zone (e.g. around a road) using connectedness with the underlying overlaid parcels.

Spatial Join – a general overlay type that attaches attributes from multiple layers (e.g. Properties, Land-use, Buildings) to a single point/feature based purely on spatial relationship (no shared key needed), producing one combined attribute table.

2.6 Clip vs. Intersect vs. Union vs. Difference – THE Comparison Table

This distinction is the single most re-tested concept in this lecture

Operation	Output geometry	Attribute handling
Clip	Only the part of A that falls inside B (B acts as a “cookie cutter”)	Only A’s attributes carry over; B’s attributes do <i>not</i> appear in the output
Intersect	Only the overlapping region of A and B	<i>Both</i> A’s and B’s attributes appear in the output
Union	<i>All</i> of A and <i>all</i> of B (every part of both)	Output contains attributes of both A and B, everywhere they apply
Difference / Erase	All of layer 1 that does <i>not</i> overlap layer 2	Only layer 1’s attributes remain
Symmetrical Difference	All areas of both inputs <i>except</i> where they overlap	Attributes of whichever input contributed that area

Memory hook: Clip = “keep A’s shape where B overlaps, throw away B’s data.” Intersect = “keep the overlap, keep *both* tables.” Union = “keep everything, keep both tables.”

2.7 Data Manipulation: Reclassify → Dissolve & Merge

Given a raw layer with fine categories (e.g. soil types A/B/C crossed with growth-potential codes d/f, giving Ad, Bd, Cf, Bf, Cd, Ad), **reclassify** collapses to the coarser categories (A, B, C only), then **dissolve & merge** removes the now-redundant internal boundaries between adjacent same-class polygons, leaving one clean polygon per class.

2.8 Buffering

Create a zone of a specified distance around a feature:

- **Buffering points** – circles around each point (multiple points → circles may merge if close enough).
- **Buffering lines** – a ribbon of constant width following the line (e.g. a river network).
- **Buffering an area** – a zone extending outward (or inward) from a polygon boundary, e.g. “area within 100 m of buildings.”

2.9 Vector Geoprocessing Tool Reference

Tool	What it does	Best use case
Buffer	Zone of specified distance around features	Proximity analysis, e.g. schools within 1 km of a road
Clip	Extract features overlapping a clip polygon	Restrict analysis to a study area boundary
Intersect	Geometric intersection common to all inputs	Overlapping zoning + flood-zone identification
Union	Combine layers, keep all features/attributes from both	Overlay admin boundaries with ecological zones
Erase	Remove areas overlapping the erase layer	Exclude protected areas from development zones
Merge	Combine datasets of the <i>same</i> geometry type into one	Combine road networks from different districts
Dissolve	Aggregate features sharing an attribute, removing internal boundaries	Turn districts into provinces
Spatial Join	Join attributes across layers by spatial relationship	Assign population data from polygons to points
Select by Location	Select features by spatial relation to another layer	Buildings within a flood zone

2.10 Case Study: Flood-Exposed Villages, Schools & Roads

Problem: identify villages/schools/road segments exposed to river flooding, to prioritise evacuation planning. **Datasets:** river, villages, schools, roads, flood zone. **Workflow:** **Buffer** the river → **Intersect**(village, buffer) to flag exposed villages → **Select** schools inside that zone → **Intersect**(road, flood zone) to flag exposed road segments → **Erase** the flood zone from usable land → **Union**(flood zone, land-use polygons) for impact-by-land-use → **Dissolve** into one continuous flood zone → **Merge** road layers sourced from different agencies into one consistent network.

2.11 Practice & Review

Q1. [MC] You overlay a school (point) layer with a flood-zone (polygon) layer using **Clip**. What happens to the flood zone’s attributes?

- (A) They appear on every clipped point
- (B) They are discarded – only the school’s own attributes remain
- (C) They replace the school’s attributes
- (D) They are averaged with the school’s attributes

Answer: (B) Clip only preserves the clipped layer’s own attributes; the “cookie cutter” layer’s attributes never appear in a Clip output (that’s what Intersect is for).

Q2. [MC] A ray cast from point *A* crosses a polygon’s boundary 3 times before leaving the map extent. Where is *A*?

- (A) Outside
- (B) Inside
- (C) On the boundary
- (D) Cannot be determined

Answer: (B) Inside – 3 is odd.

- Q3. [Short Answer]** Give one example each of a spatial relationship that (a) can and (b) cannot be computed purely from coordinates. *Answer:* (a) “Point is contained in polygon” – fully computable from coordinate geometry. (b) “Road segment X intersects railway segment Y” as a real at-grade crossing vs. an overpass – coordinates alone tell you the *lines* cross, not whether the real-world features actually meet (needs an attribute like “bridge/underpass”).
- Q4. [Short Answer]** You need every part of two overlapping zoning layers preserved, along with both layers’ attributes everywhere. Which overlay operation, and why not Clip? *Answer: Union.* Clip would discard the second layer’s attributes and restrict output geometry to only the overlap with the clip feature; Union keeps the full extent of *both* inputs and both attribute tables.
- Q5. [Essay]** Design the vector-overlay workflow to answer: “Which residential parcels are within 500 m of a river AND fall inside a designated flood-risk polygon?” *Answer:* (1) **Buffer** the river line by 500 m. (2) **Intersect** the buffer with the flood-risk polygon layer (AND logic – must satisfy both conditions) to get the combined risk zone, preserving both layers’ attributes. (3) **Select by Location** (or **Intersect**) the residential-parcel polygon layer against that combined risk zone to extract only the qualifying parcels. (4) Optionally **Dissolve** the output by parcel ID if overlay fragmented any parcel into slivers.

3 18 Sept: Raster Data Analysis – Part 1

3.1 Overview

Raster GIS operations are efficient for **overlaying** many thematic layers at once, allow **mathematical operations** across layers cell-by-cell, provide a natural frame for **surface modelling/dispersion**, and enable **3D visualization**.

3.2 Raster Functions/Operations – The Big Picture

- **Map Algebra** – mathematical operations across multiple raster layers.
- **Neighbourhood functions** – **Euclidean** (distance), **Metric** (distance-based), **Topological** (adjacency-based, e.g. flow direction/accumulation for a river network).

Map Algebra operator families

Arithmetic: + - * / ^ **Boolean:** True/False **Comparison:** < <= > >= **Logical:** contained-within-sets **Combinatorial:** overlay of different grids with attributes
Accumulative: sum/divide/multiply groups of values **Assignment:** assigning values during operations

3.3 Raster-Based Overlay

- **Simple addition:** cell-by-cell sum of scored resource grids $A + B + C =$ a composite grid (e.g. scores $1+3+2=6$ at one cell).
- **Boolean combine:** a two-phase process – *geometric phase* (common grid framework ensures every pixel overlays at the same position) then *attribute phase* (a threshold decides “suitable/not”, and a combination rule – AND/OR – produces the final result).

Worked example: Environmental Contamination Risk Index

Three raw rasters – Heavy Metal Concentration (0–500 mg/kg), Soil Permeability (1–5), Terrain Slope (0–20%) – are first **normalized** to 0–1 (divide by their max: $HMC/500$, $Perm/5$, $Slope/20$), then combined via a **weighted formula**:

$$\text{Risk Index} = 0.4 \cdot HMC_{norm} + 0.3 \cdot Perm_{norm} + 0.3 \cdot Slope_{norm}$$

producing a single Risk Map (red = high risk, green = low risk). **This normalize-then-weight-then-sum pattern is the template for almost any multi-criteria raster overlay question.**

3.4 Overlay: The 4 Basic Rules

1. **Enumeration Rule** – every unique combination of input attribute values is preserved in the output (nothing is collapsed).
2. **Dominance Rule** – one value “wins”; the operation picks a single value from several inputs (e.g. max, or “highest bidder” – applied to ratio data).

3. **Contributory Rule** – each attribute value contributes to the result via an operation like addition (works for nominal categories, e.g. summing 0/1 presence layers into a count layer).
4. **Interaction Rule** – a *pair* of values from two inputs jointly determines the output (via a lookup/legend of all possible value-pairs, e.g. “1&1”=1, “1&2”=2, “2&1”=4, etc.) – decisions at each step of a combination may differ.

Real-world use

Landslide-susceptibility mapping (e.g. Sikkim Himalaya) combines rainfall, earthquake magnitude, slope, distance-to-river, topographic wetness index, stream power index, and land-cover rasters – exactly this kind of multi-layer weighted/interaction overlay – into one Very-Low→Very-High susceptibility zone map.

3.5 Cell/Pixel/Grid-Based Operation Classification

The single most-repeated classification question in this course

Type	Depends on	Example
Local	Only the cell's own value(s) across one or more <i>aligned</i> input grids – no neighbours involved	<code>new_map = old_map_1 + old_map_2</code> ; re-classifying a single cell's land-use code
Focal	The cell x , its own attribute $f(x)$, and the attributes of its neighbourhood $n(x)$	3×3 mean filter; counting buildings within 5 km of a fire station
Zonal	All cells that belong to the same <i>zone</i> (group of cells with the same value or region)	Average slope per watershed
Global (Incremental)	The <i>entire</i> raster grid	Euclidean/cost-distance from a source to every other cell; least-cost path

Trick to remember: Local = pixel; Focal = a small neighbourhood (kernel) around the pixel; Zonal = a whole named region; Global = the whole dataset.

3.6 Local Functions

Definition: operate on the values of all attributes relating to *one* cell location, independent of neighbouring cells; generic form $U = f(X_1, X_2, \dots)$ producing a new grid from one or more input grids.

- **Single-raster statistics** (numeric data): max, min, range, sum, mean, median, standard deviation.

- **Single-raster statistics** (numeric *and* categorical data): majority, minority.
- **Multiple-raster local functions:** Average/mean – `outgrid = mean(ingrid_1, 2*ingrid_2, (ingrid_3+ingrid_4));` Merge – fills NODATA cells of one grid with the corresponding value from another (`if X1==NODATA then U=X2 else U=X1`); Maximising/Minimising – `U = max(X1, X2, ...)` or `min(...)`, cell-by-cell across the stack.
- **Multiply by a constant** (e.g. every cell $\times 3$) vs. **multiply by another grid** (cell-by-cell product) – both are local operations, just with a scalar vs. a grid as the second operand.
- **Combine** – a non-statistical local operation that assigns a unique output value to every unique *combination* of input values (e.g. combining slope class + aspect class into one composite class code).

3.7 Reclassification

1. **Arbitrary reclassification** – a lookup table maps each input value to an arbitrary new output value:

```
if X==a1 then U=b1; else if X==a2 then U=b2; ...; else U=X
or equivalently: outgrid = con(ingrid==1, 5, con(ingrid==2, 4, con(ingrid==3,
2)))
```

2. **Boolean reclassification** – combines *two* input grids via a coding-scheme lookup table (rows = grid 1 values, cols = grid 2 values, cell = output code or “N” for NODATA):

```
if X1==a1 and X2==b1 then U=c1; else if ...; else U=NODATA
```

3. **Statistical reclassification** – **Slice** – divides the value range into zones either in **equal intervals** (equal-width value ranges, so zone sizes on the ground vary) or **equal areas** (each output zone covers the same land area, so value-range widths vary): `outgrid = slice(ingrid, EQAREA|EQINTERVAL, nzones, base_zone#, in_min, in_max).`

Advantages of reclassification: (1) creates a simplified raster (e.g. collapsing continuous slope values into a few classes); (2) creates unique category values (e.g. “irrigated cropland” = 1); (3) creates a ranking of cell values (e.g. 1=least suitable to 5=most suitable).

3.8 Practice & Review

- Q1. [MC]** “Deriving a map of surface waterbodies by reclassifying lakes/ponds and rivers into one class” is an example of which raster operation type?

- (A) Focal
- (B) Zonal
- (C) Local
- (D) Global

Answer: (C) Local – each cell’s new value depends only on its own current class code (a lookup-table reclassification), not on neighbours or the whole grid.

Q2. [MC] “Creating a shortest-distance map from two source locations across a raster” is:

- (A) Local
- (B) Focal
- (C) Zonal
- (D) Global

Answer: (D) Global – computing distance to a source from *every* cell in the grid requires processing the entire raster, the defining trait of a global/incremental function.

Q3. [Short Answer] Distinguish the Dominance rule from the Contributory rule in raster overlay, with an example of each. *Answer:* Dominance picks a single winning value from multiple inputs (e.g. `max` of four bid-value rasters → “highest bid” raster) – appropriate for ratio data. Contributory lets every input value add to the result (e.g. summing four binary 0/1 hazard-presence rasters into a 0–4 “hazard count” raster) – appropriate for nominal/categorical presence data.

Q4. [Short Answer] Why normalize raster layers to 0–1 before a weighted overlay? *Answer:* Raw layers have different units and ranges (e.g. 0–500 mg/kg vs. 0–20%); without normalizing, whichever layer has the largest raw numeric range would dominate the weighted sum regardless of its assigned weight, defeating the purpose of weighting by importance.

Q5. [Essay] You have a slope raster (0–20%) and want 5 output zones such that each zone covers roughly equal land area. Which reclassification tool, and how does it differ from the alternative? *Answer:* Use **Slice** with the **EQAREA** option (`outgrid = slice(ingrid, EQAREA, 5, ...)`). **EQAREA** computes zone boundaries so that each of the 5 output classes contains an (approximately) equal *count of cells* (hence equal area), meaning the underlying value-range widths will differ across zones. This differs from **EQINTERVAL**, which instead divides the 0–20% value range into 5 equal-width bins (each 4% wide) regardless of how many cells/area fall into each bin – appropriate when you care about equal value resolution rather than equal spatial coverage.

4 19 Sept: Raster Data Analysis – Part 2

4.1 Overview

Continuation of the Local/Focal/Zonal/Global classification (Section 3): this lecture goes deep on **Focal**, **Zonal**, and **Global** operations.

4.2 Focal Functions (Neighbourhood)

Definition: relate a cell to its neighbours; the surrounding cells are chosen by their **distance** and/or **directional** relationship to the focal cell; the result is written to the *focal (centre) cell* of the output grid, and the window then slides to the next cell.

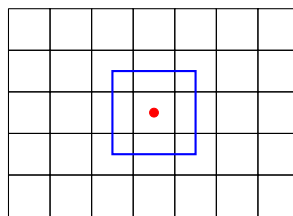
Worked mini-example (neighbourhood values 1, 3, 4, 6, 4)

Total = 18; Average = 4 (rounded); Minimum = 1; Maximum = 6; Minority = 1 (or 3, or 6 – least frequent); Majority = 4 (most frequent); Variety = 4 (distinct values present); Median = 4; Deviation = 0; Std. dev. = 2; Proportion = 40.

Every focal function needs 3 parameters: (1) target location(s) – the neighbourhood focus; (2) a specification of the neighbourhood shape/size around each target; (3) a function to run on the elements inside that neighbourhood.

Common neighbourhood shapes:

- **Rectangle** – e.g. a 4×4 block of cells.
- **Circle** – an isotropic (direction-independent) disc of radius r .
- **Annulus** – a ring (donut) between an inner and outer radius, excluding the immediate centre.
- **Wedge / Pie** – a directional sector, useful for direction-dependent analyses (e.g. wind exposure from one bearing).



A 3×3 rectangular kernel centred on the focal cell

4.3 Neighbourhood Analysis / Spatial Filtering

A **spatial convolution / filter** replaces each cell's value with some function of itself and its surrounding cells, using a **kernel** of weights that must sum to 1.0 for a smoothing filter.

- **Low-frequency (low-pass) filter – Mean filter:** kernel of all 1/9 for a 3×3 window; replaces each cell with the neighbourhood mean; smooths data; a larger window smooths more.

- **Median filter:** uses the neighbourhood *median* instead of the mean – smooths while being robust to extreme outliers.
- **High-frequency (high-pass) filter – negative-weight (edge-detection) filter:** exaggerates rather than smooths local detail; used for edge detection. Example 3×3 kernel: centre weight $+9$, all 8 neighbours -1 .
- **Standard-deviation (texture) filter:** computes the SD of neighbourhood values; **high SD = high texture/variability** (e.g. a boundary/edge cutting through the window), **low SD = low texture** (a homogeneous patch); also used for edge detection.

Worked edge-detection kernel example

Kernel $w_i = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{pmatrix}$. For a window straddling an edge with values 2 on one side and 5 on the other, e.g. centre=5 with one neighbour=5 (weight 9) and remaining neighbours=2 (weight -1 each): $1(5)(9) + 5(2)(-1) + 3(5)(-1) = 45 - 10 - 15 = 20$ (illustrative – the key exam skill is plugging values $\times$ weights into $\sum f_i v_i w_i$, not memorizing this exact number).

Convolution $\approx$ Focal, generalized

The convolution operation used in CNNs is explicitly presented as **an extension of the focal operation**: a learned kernel slides over a grid exactly like a focal-mean/edge filter, except the weights are learned rather than fixed.

Worked focal-function problem (fire station example): *Question*: how many residential buildings lie within 5 km of a given fire station? **Target** = the fire station; **Neighbourhood** = the area within a 5 km radius (a circular kernel); **Function** = count residential buildings.

4.4 Zonal Functions

Definition: operate on a group of cells sharing the same value or belonging to the same named region, called a **zone**. Zones may be **contiguous** (spatially connected) or **non-contiguous** (separate patches sharing one class, e.g. “Zone 3” appearing in two disconnected places).

Characteristics: zonal functions do *not* change the **boundaries** of regions; they *change the attribute value* assigned to each region based on its statistics or a user specification. Useful for spatial distribution understanding, quantitative shape measurement, and spatial-association statistics.

A typical zonal function needs **two grids**: a **zone grid** (defines the size/shape/location of each zone) and a **value grid** (the raster actually being summarised per zone).

Worked example: Zonal Area

Given a 4×4 zone grid with 4 numbered zones and a database definition where 1 cell = 250,000 m²: `outgrid = zonalarea(ingrid)` reassigns every cell in Zone 4 (say, 7 cells) the value $7 \times 0.25 \text{ km}^2 = 1.75 \text{ km}^2$ – i.e. every cell in a zone gets the *same* value, equal to that zone’s total area.

Applications: measuring zonal geometry (area, perimeter) for landscape-ecology studies; comparing statistics between two rasters (e.g. mean *slope* per *watershed* – slope is the value grid, watershed is the zone grid); defining election wards/administrative sub-regions as zones for aggregate reporting.

4.5 Global (Incremental) Functions

Definition: the output cell value depends on processing the **entire** grid, not a local neighbourhood or a single zone.

Examples: computing distance from one cell (or group of cells) to *all* other cells; weighting that distance by an impedance/friction factor to get **cost-distance surfaces**. **Uses:** diffusion modelling, shortest-path modelling, distance-based site selection, visibility analysis, connectivity/fragmentation in habitat analysis.

4.5.1 Proximity

Computes, for every cell, an attribute equal to the length of the shortest path to a target location/area, measured either as:

- **Euclidean distance** – straight-line distance, ignoring obstacles.
- **Cost distance (“weighted distance”)** – distance modified by a friction/impedance surface; the **least-cost path** is the minimum-cost route, which is often *not* the same as the shortest Euclidean path.

Barrier effects on travel zones (concentric rings from source A to target B):

- **No barrier** – perfectly circular (Euclidean) rings.
- **Absolute barrier** – rings cannot cross the barrier at all; the path must go all the way around it.
- **Partial barrier (friction)** – rings can cross the barrier but are “pushed” inward/slowed, so the barrier region shows compressed/denser rings (higher local cost) rather than being impassable.

Spread Functions: building a friction (travel-time) surface

A land-use raster (Cropland, Road, Rangeland, Quarry) is converted cell-by-cell into a travel-time raster using domain knowledge of typical speeds on each land-use type (e.g. Road → fastest travel time value; Quarry → slowest/blocked). This friction surface is then the input to a cost-distance/least-cost-path global function.

Raster buffering, generalized: instead of a constant-radius buffer, vary the buffered distance according to an **accessibility layer** – e.g. “areas reachable within 5 minutes” vs. “within 10 minutes” of a city centre, which will *not* be a perfect circle if travel speed varies spatially (this is buffering built on top of a cost-distance global function, not a simple Euclidean buffer).

4.6 Practice & Review

Q1. [MC] A 3×3 mean filter kernel must have weights that:

- (A) Sum to 0
- (B) Sum to 1.0
- (C) Are all negative
- (D) Sum to the kernel size squared

Answer: (B) Weights must sum to 1.0 (e.g. nine cells of $1/9$ each) so the filter preserves the overall magnitude/brightness while smoothing.

Q2. [MC] High standard deviation in a focal SD (texture) filter window indicates:

- (A) A perfectly flat, homogeneous area
- (B) An edge or highly variable/textured area
- (C) Missing data
- (D) The centre of a large uniform zone

Answer: (B)

Q3. [Short Answer] Why does a zonal function require *two* rasters, and what happens to the zone boundaries after running it? *Answer:* It needs a *zone raster* to define which cells belong to which region and a separate *value raster* supplying the numbers to be summarised. The zone boundaries never change – only the attribute value assigned within each zone is updated (e.g. every cell in Watershed 3 gets set to the mean slope computed over Watershed 3’s footprint).

Q4. [Short Answer] Explain why cost-distance and Euclidean distance can produce different “nearest” answers for the same two points. *Answer:* Euclidean distance measures straight-line ground distance regardless of terrain. Cost distance accumulates a friction/impedance value along the path (e.g. steep slope, dense forest, water = high cost), so the cell that is geometrically closest may not be the cheapest/fastest to reach if the straight path crosses high-friction terrain – the least-cost path can detour around obstacles, making it longer in Euclidean terms but cheaper overall.

Q5. [Essay] A municipal planner wants to combine (a) the number of hospitals within 5 km of every residential cell (focal) with (b) the average income level per administrative ward (zonal) to prioritise ambulance-station siting. Describe a realistic combined workflow. *Answer:* (1) Run a **focal** function over the hospital-presence raster with a 5 km circular neighbourhood and a **count/sum** function, producing a “hospital access score” raster where each cell holds the count of hospitals in its neighbourhood. (2) Run a **zonal** function using the administrative-ward polygon/raster as the zone grid and an income raster as the value grid, computing mean income per ward, then assign that mean back to every cell in the ward (so every cell in a ward shares one income value). (3) Combine the two rasters with a **local** map-algebra operation (e.g. a weighted sum or the Contributory rule) to produce a single “priority score” raster identifying underserved, lower-income areas as top candidates for new ambulance stations. This mirrors the GS2.401 exam pattern of “give a real example combining both focal and zonal operations.”

5 Previous-Year Question Bank (Filtered & Adapted)

Source & filtering note

Adapted from two previous mid-semester papers by the *same instructor*: **GS2.401 Mid-Sem, 25 Sept 2025** (Monsoon 2025 – very close topical overlap with this quiz) and **GS2.401 Mid-Sem, 21 Sept 2024**. Questions on **map projections, Geoid/DGPS, and raw raster storage formats (BIP/BSQ/BIL)** were in these papers but are **not on your 4–19 Sept slides**, so per your instruction they are **omitted here** – do not expect them on this quiz. What remains below is reworded/adapted, not verbatim, and matches the instructor’s characteristic **multi-select** and **scenario/essay** style.

5.1 Multi-Select Style (2025 paper Part I) – watch for booby-trap options

Q1. [Multi-select] Select all *valid* statements about vector data structures:

- (a) The spaghetti model preserves topological relationships between spatial objects.
- (b) Polygon-arc topology inherently makes sure no two polygons overlap.
- (c) Arc-polygon topology captures the adjacency relationship between two polygons.
- (d) A vertex dictionary stores topological relationships by avoiding duplication of vertices.

Answer: (c) and (d). (a) is **false** – the spaghetti model stores geometry with *no* topology at all (every line is an independent string of coordinates with no shared-node information); that is precisely why topology-building (arc-node lists, snapping) is needed on top of it, as covered in Section 1. (b) is a common trap: arc-node/polygon topology *enforces consistent shared boundaries*, but “inherently making sure no two polygons overlap” overstates it – overlaps can still be digitized if boundaries are not properly built; topology validates and can *flag* overlaps via rules, it doesn’t automatically forbid bad input. (c) is true – arc-polygon (left-right) topology is exactly how adjacency between two polygons is derived. (d) is true – a vertex dictionary is a de-duplication structure that underlies clean topology.

Q2. [Multi-select] Which of the following represent *topological* properties (i.e. survive a stretch/distortion of the map)?

- (a) Campus area is exactly 66 acres.
- (b) Road A is connected to the entry gate.
- (c) The canteen is located south-east of the football ground.
- (d) A new construction site exists inside the campus.

Answer: (b). “Connected to” is a topological (adjacency/connectivity) relationship that is scale-and-distortion-invariant. (a) is a *metric* property (area), which **changes** under stretching – not topological. (c) is a *directional/relative-position* property, which is generally treated as *not* guaranteed topological (it can flip under certain distortions) – excluded per the slides’ list of what changes (distance, angles, relative proximities). (d) is a factual/attribute statement, not a topological relationship at all.

Q3. [Multi-select] By digitizing agricultural farm boundaries in a village, which topological constraint would you *deliberately choose not to enforce*?

- (a) Two neighbouring farms can overlap.
- (b) Two neighbouring farms cannot overlap.
- (c) A boundary segment can be shared by at most two neighbouring farms (including itself).
- (d) Corners may belong to more than two neighbouring farms.

Answer: (a). You would *not* allow farms to overlap – overlap is precisely the topology error rule you enforce *against* (rule (b) is the one you **DO** apply). (c) and (d) are legitimate, correctly-applied topology rules (a shared edge borders exactly 2 polygons; a shared corner/node can legitimately touch 3+ polygons).

Q4. [Multi-select] For a metro-network map showing preceding/succeeding stations, select all valid statements:

- (a) The metro map is topologically equivalent to the actual network on the ground.
- (b) The metro map correctly represents the true distance between two stations.
- (c) Connectedness between two stations is often misrepresented in the metro map.
- (d) A curved line on the ground (like “S”) can be drawn as a straight line in the map.

Answer: (a) and (d). A metro diagram is the classic real-world example of **pure topology without metric accuracy**: it preserves connectivity/order (a) and freely straightens curves (d) precisely *because* distance/shape are non-topological and can be sacrificed (b is false), while connectivity is topological and therefore *preserved*, not misrepresented (c is false – the diagram may misrepresent geometry, but it is deliberately built to *correctly* preserve connectivity/order, which is its entire point).

Q5. [Multi-select] Select all valid statements for a raster model of spatial data:

- (a) A raster model better describes discrete geographic objects (like parcel boundaries).
- (b) A focal operation may use grid-cell values from a 3×3 neighbourhood of a target cell.
- (c) A raster map with a “bigger numerical value” inherently provides more spatial detail.
- (d) Temperature, precipitation, and elevation are better represented by a raster model.

Answer: (b) and (d). (a) is false – *vector* better describes discrete, sharply-bounded objects; raster is better for **continuous surfaces** (which is exactly why (d) is true – temperature/precipitation/elevation are continuous fields). (c) is a trap confusing cell *value* with cell *resolution* – resolution (smaller cell size) determines detail, not the magnitude of the number stored in a cell.

5.2 Scenario / Overlay-Type Identification (2025 paper Part II)

Q1. [Short Answer] State the spatial overlay type for: “district-wise (polygon) count of health-care facilities (point locations).” *Answer: Point-in-polygon overlay* – each health-care point is tested for containment in a district polygon, then aggregated by `COUNT(...)` `GROUP BY district` (same pattern as the well/county example in Section 1).

Q2. [Short Answer] State the overlay type for: “finding road segments (line) passing through forest areas (polygon).” *Answer: Line-in-polygon overlay* (see the road/soil-type worked example, Section 2) – each road line is split at forest-polygon boundaries and the resulting segments inside the forest are selected/tagged.

- Q3. [Short Answer]** State the overlay type for: “creating a new polygon layer where each feature carries both land-use and soil-type attributes.” *Answer:* **Polygon overlay** (specifically a **Union** or **Intersect**, depending on whether areas outside the overlap should be retained) – this is the landuse×soil worked example in Section 2.
- Q4. [Short Answer]** Classify into Local/Focal/Zonal: (a) deriving a “surface waterbodies” map by reclassifying lakes/ponds/rivers into one class; (b) creating a shortest-distance map from two source points. *Answer:* (a) **Local** (a per-cell reclassification/lookup, no neighbourhood or whole-grid dependency). (b) **Global** – note the original 2025 paper’s hint suggested “focal,” but per the slide definitions in Section 3–4, computing distance-from-source across an *entire* grid is the textbook definition of a **Global (incremental)** function, since it depends on processing the whole grid, not a bounded local neighbourhood. If your instructor’s rubric specifically wants “focal,” be ready to justify *Global* with the “depends on processing the entire grid” slide definition – this is a genuine terminology edge case worth flagging to your TA if it recurs.
- Q5. [Essay]** Give a real-world example that combines both **focal** and **zonal** operations, specifying the input rasters and how they combine. *Answer:* See the worked ambulance-siting example at the end of Section 4 (hospital-count focal raster + mean-income zonal raster, combined via local map algebra).
- Q6. [Essay]** After detecting waterbodies via NDWI + simple thresholding and polygonizing, spurious tiny (single-pixel) polygons appear. What causes this, and how would you fix it? What other index could improve detection? *Answer:* **Cause:** sensor/random noise (speckle) near the water/land threshold boundary, or mixed pixels straddling the true edge, get misclassified as isolated “water” pixels when a hard threshold is applied – exactly the **sliver-polygon / random-noise** problem from Section 1. **Remedy:** apply a **focal majority filter** (Section 4) before polygonizing to remove salt-and-pepper misclassification, and/or set a **minimum-area threshold** to discard polygons below one/two pixels’ area during the vector **data-cleaning** pass (removing dangling/sliver artifacts, Section 1). **Other index:** **NDBI** or a combined **MNDWI** can supplement NDWI in built-up-adjacent water; more simply, cross-checking with a **slope** or **elevation** raster (water tends to sit in local minima) via a **local AND/Boolean combine** (Section 3) reduces false positives.
- Q7. [Essay]** Argue whether the vector model is sufficient or insufficient for: (a) representing house footprints; (b) representing population density across an urban area marked as residential/industrial/commercial. *Answer:* (a) **Vector is sufficient (and preferred)** – house footprints are discrete, sharply bounded objects with clean edges; a polygon vector layer captures exact boundaries with no loss, matching the “vector better describes discrete objects” principle from the multi-select trap above. (b) **Vector alone is insufficient** – population density is a *continuous field* that varies within a single zoning polygon; forcing it into vector zones (residential/industrial/commercial) discards all intra-zone variation. A **raster** representation (e.g. a population-density grid, possibly derived via a **zonal** operation that distributes a zone’s total population across its raster cells, or a dasymetric/interpolated surface) is more appropriate, consistent with “raster is better for continuous surfaces” (elevation/temperature/precipitation analogy).

5.3 Applied/Design Questions (2024 paper, topics in scope)

- Q1. [Essay]** List and briefly explain the errors that occur in *manual digitization* of maps. *Answer:* Draw directly from Section 1: **node errors** (undershoot/overshoot from inadequate snap tolerance); **pseudonodes** (redundant vertex splitting one line into two arcs); **dangling nodes** (an unconnected line-end that should have closed a polygon); **sliver polygons** (from re-digitizing what should be an identical shared boundary); **missing label points** (polygon has no attribute anchor); **line crossings with no node** (intersection not properly noded, breaking connectivity).
- Q2. [Essay]** A dataset of bank-branch point locations covers 500 sq. km. Show how you would identify branches within 10 km of each other so none serve overlapping populations. *Answer:* **Buffer** every branch point by 5 km (half the 10 km exclusion distance, so that two buffers overlap exactly when the branches are <10 km apart) → **Intersect/Select by Location** to find pairs of buffers that overlap each other → for each overlapping pair, flag one branch as a candidate to relocate. Alternatively (and more directly), compute a **proximity/Euclidean-distance** global function from every branch to all others, then **Select** pairs with distance <10 km – either way this is a direct application of Section 2’s Buffering and Section 4’s Proximity concepts.
- Q3. [Essay – HYDRA-style buffer-zone scenario]** A water body’s protective buffer is 9 m (if spread area <10 Ha) or 30 m (if spread area ≥10 Ha), based on its full-tank-level (FTL) boundary from an old paper topo-sheet (1:25,000, geographic coordinates) plus a 10 m-resolution classified satellite water/land mask (UTM projection, 3 recent high-rainfall years). Explain how you would (A) build a system to tell the public whether a plot lies in the prohibited zone, and (B) count existing buildings inside that zone. *Answer:* **Pre-processing: georeference** the scanned paper topo-sheet (Section 1 – GCPs, RMS check) into the same UTM projection as the satellite-derived water masks; **vectorize** both the FTL boundary (from the topo-sheet) and each year’s classified water polygon (raster→vector, watching for sliver/dangling errors); take the **Union (max extent)** of the 3 years’ water polygons as the operative water-spread area (since the buffer should be conservative across recent high-rainfall years) and compute its area to decide 9 m vs. 30 m per water body. **(A): Buffer** each water-body polygon by the appropriate distance → this buffer layer is the prohibited zone; for any queried plot, run a **point/polygon-in-polygon (contained-in) spatial query** against the buffer layer to answer yes/no. **(B): Intersect** (or **Select by Location**) the building-footprint polygon layer against the buffer zone, then **COUNT** the selected buildings – exactly the “is contained in → store as attribute → aggregate/count” pattern from Section 1.
- Q4. [Essay – pipeline network scenario]** Centerline road-segment data (start/end points, width 8–48 m, road name, possibly duplicated names) needs to become (A) a pipeline vector dataset offset 1 m from the left road boundary, and (B) a junction/turn vector dataset with junction-type attributes. *Answer:* **Data model:** the surveyed input is a simple line (centerline) **vector** dataset in a **non-topological (spaghetti-like) storage model** (independent segments, duplicate names, no shared-node information yet). It must first be converted into a proper **arc-node topology** model (Section 1) – snapping segment endpoints into shared nodes so that connectivity/junctions are explicit and traceable, resolving duplicate-name ambiguity by unique segment IDs rather than names. **(A):** compute each segment’s **road boundary** by offsetting the centerline by half its width (using the width attribute), then generate the pipeline line as a further 1 m offset from that left boundary – a **buffer/offset (transform-**

mation) operation per segment, storing pipe diameter as an attribute and explicit start/end coordinates. **(B)**: once arc-node topology is built, every **node** where ≥ 3 arcs meet (or where two arcs meet at an angle) is a candidate **junction/turn**; extract these nodes as a point layer and classify each by the number/angle of connecting arcs (junction-type attribute) – a direct use of **connectivity** (Section 1's 5 topological relationship types) to programmatically discover junctions rather than manually digitizing them.

6 Master Cheat Sheet

One-page formula/definition dump

Point-in-polygon (ray-casting): odd # of edge crossings = inside; even = outside.

RMS error: $\text{RMS} = \sqrt{\frac{1}{N} \sum (\Delta X_i^2 + \Delta Y_i^2)}$; target <1 pixel.

Line intersection: $x_i = -\frac{a_1 - a_2}{b_1 - b_2}$, $y_i = a_1 + b_1 x_i$; check x_i within both segments' range.

Mean (low-pass) 3×3 kernel: all weights = 1/9 (sum to 1.0).

Edge-detection (high-pass) 3×3 kernel: centre +9, 8 neighbours −1 each.

Weighted overlay pattern: normalize each layer to [0, 1] → multiply by importance weight → sum.

Zonal area: every cell in a zone gets the value = (# cells in zone) × (area per cell).

Scale: RF = map distance : ground distance. Large scale (1:12k–1:25k) = small area, high detail. Small scale (1:250k+) = large area, coarse detail.

	Vector overlay (Sec. 2)	Raster classification (Sec. 3–4)
Smallest scope	Clip – only clip-feature attributes kept	Local – one cell, no neighbours
Overlap only, both tables	Intersect	–
Everything, both tables	Union	–
Neighbourhood-dependent	(buffer is closest analogue)	Focal – kernel/window
Region-dependent	Dissolve (merges same-attribute adjacents)	Zonal – named zone, 2 grids needed
Whole-dataset-dependent	(Warping/rubber-sheet, whole-layer transform)	Global – distance/cost surfaces

5 Types of Topological Relationships

Connectivity · Adjacency · Containment · Proximity · Relative Direction

4 Raster Overlay Rules

Enumeration (keep every combo) · Dominance (one value wins) · Contributory (each value adds) · Interaction (value-pairs looked up)

10 Map Generalization Operators

Simplification · Smoothing · Collapse · Aggregation · Amalgamation · Merging · Refinement · Exaggeration · Enhancement · Displacement

7 Rapid Self-Test

- Q1.** What is a pseudonode, and which tool removes it? *[A redundant same-feature vertex splitting one line into two arcs; removed via Split/Unsplit.]*
- Q2.** Name the three ways to express map scale. *[Word statement, Representative Fraction, Graphic scale.]*
- Q3.** What does the odd-even rule in point-in-polygon testing determine? *[Whether the query point is inside (odd crossings) or outside (even crossings) the polygon.]*
- Q4.** Which overlay operation discards the “cookie-cutter” layer’s own attributes? *[Clip.]*
- Q5.** Which raster operation type does NOT change zone boundaries, only attribute values? *[Zonal.]*
- Q6.** Give the generic form of a local raster function. *[$U = f(X_1, X_2, \dots)$.]*
- Q7.** What is the difference between Interpolation and Extrapolation? *[Interpolation fills gaps between two known bounding values; extrapolation predicts beyond the known range, with only one side known.]*
- Q8.** Which map-generalization operator replaces an area object with point+line objects? *[Collapse.]*
- Q9.** What are the 3 required parameters of any focal function? *[Target location(s), neighbourhood specification, function to apply.]*
- Q10.** Which raster overlay rule is appropriate for combining several binary presence/absence layers by simple addition? *[Contributory rule.]*
- Q11.** What causes “scan skew” in satellite imagery? *[Variation between the mirror’s actual angular velocity and its nominal velocity during scanning.]*
- Q12.** Why is a well-distributed (not clustered) set of GCPs important? *[Clustering biases the transformation model toward accuracy near the cluster and poor accuracy elsewhere – especially at image edges.]*
- Q13.** What is the key difference between a friction surface’s “absolute barrier” and “partial barrier”? *[Absolute barrier blocks all passage (rings must detour around it entirely); partial barrier allows passage but at increased cost (rings compress/slow through it).]*
- Q14.** Between Clip and Intersect, which one keeps attributes from BOTH input layers? *[Intersect.]*
- Q15.** What is the defining feature that makes an operation “Global/Incremental” rather than “Focal”? *[It depends on processing the entire grid, not a bounded neighbourhood around a single target cell.]*